

☐☐ OpenOCD & ARM GDB Cheatsheet for ARM Embedded Development

☐☐ Table of Contents

1.
 - ☐☐ Quick Reference Card
 - o Essential Tools Setup
2.
 - ☐☐ Quick Build → Flash → Debug Workflow
 - o 1. Build (Generic ARM Project)
 - o 2. Flash (One-liner)
 - o 3. Debug (Launch)
3.
 - ☐☐ OpenOCD Command Reference
 - o Hardware Interface Files
 - o Target Configuration Files
 - o Common OpenOCD Commands
4.
 - ☐☐ ARM GDB Command Cheatsheet
 - o Connection & Basic Control
 - o Breakpoints & Stepping
 - o Memory & Registers
 - o Variables & Watchpoints
5.
 - ☐☐ Flash & Debug Using Monitor Commands
 - o Complete Flash Sequence
 - o Step-by-Step Flash Commands
 - o Debug Session Commands
6.
 - ☐ Power User Commands
 - o Advanced Monitor Commands
 - o Useful GDB Scripts
 - o OpenOCD Telnet Interface
7.
 - ☐☐ Target-Specific Examples
 - o STM32F4 Discovery
 - o Infineon CYT2CL7BAAQ (TRAVEO™ T2G Cluster)
 - o CYT2CL7BAAQ Flash Programming
 - o CYT2CL7BAAQ Memory Map
 - o Generic Cortex-M Setup

8.
 - Zephyr RTOS Support for CYT2CL7BAAQ
 - o Board Configuration
 - o West Commands for CYT2CL7BAAQ
 - o Custom OpenOCD Runner Configuration
 - o OpenOCD Configuration for Zephyr
 9.
 - Essential File Structure
 - o Bare Metal Project
 - o Zephyr RTOS Project
 10.
 - Common Issues & Solutions
 - o Connection Issues
 - o Flash Programming Issues
 - o Debugging Issues
 11.
 - Quick Command Reference Table
 12.
 - Pro Tips
-

□□ Quick Reference Card

Essential Tools Setup:

- **OpenOCD**: Hardware abstraction for debugging/flashing
 - **arm-none-eabi-gdb**: Cross-platform GDB for ARM Cortex
 - **arm-none-eabi-gcc**: ARM cross-compiler toolchain
-

□□ Quick Build → Flash → Debug Workflow

1. Build (Generic ARM Project)

```
# Using CMake + Ninja
mkdir build && cd build
cmake -DCMAKE_TOOLCHAIN_FILE=arm-none-eabi.cmake ..
ninja
```

```
# Using Makefile
make clean && make all
```

```
# Using arm-none-eabi-gcc directly
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -O2 -g \
-T linker_script.ld src/*.c -o firmware.elf
```

2. Flash (One-liner)

```
# Direct flash with OpenOCD
```

```
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg \  
-c "program firmware.elf verify reset exit"
```

3. Debug (Launch)

Terminal 1: Start OpenOCD server

```
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg
```

Terminal 2: Connect GDB

```
arm-none-eabi-gdb firmware.elf -ex "target remote :3333"
```

❏❏ OpenOCD Command Reference

Hardware Interface Files:

```
# ST-Link (STM32 Discovery/Nucleo boards)  
-f interface/stlink.cfg
```

```
# J-Link  
-f interface/jlink.cfg
```

```
# CMSIS-DAP  
-f interface/cmsis-dap.cfg
```

```
# Black Magic Probe  
-f interface/bmp.cfg
```

Target Configuration Files:

```
# STM32F4 series  
-f target/stm32f4x.cfg
```

```
# STM32F1 series  
-f target/stm32f1x.cfg
```

```
# STM32L4 series  
-f target/stm32l4x.cfg
```

```
# Generic Cortex-M4  
-f target/stm32f4x.cfg
```

```
# nRF52 (Nordic)  
-f target/nrf52.cfg
```

Common OpenOCD Commands:

```
# Start OpenOCD with config  
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg
```

```
# One-shot programming  
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg \  
-c "program firmware.elf verify reset exit"
```

```
# Custom script execution  
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg \  
-c "init; reset halt; flash write_image erase firmware.bin 0x08000000; reset run; exit"
```

☐☐ ARM GDB Command Cheatsheet

Connection & Basic Control:

```
# Connect to OpenOCD
target remote localhost:3333
# or
target extended-remote localhost:3333

# Load program
load
# or
load firmware.elf

# Reset target
monitor reset halt
monitor reset run

# Continue execution
continue
c

# Halt execution
monitor halt
# or Ctrl+C
```

Breakpoints & Stepping:

```
# Set breakpoints
break main
break *0x08001234
break filename.c:42

# List/delete breakpoints
info breakpoints
delete 1
clear main

# Stepping
step      # Step into functions
next      # Step over functions
stepi     # Single instruction step
nexti     # Next instruction
finish    # Step out of function
```

Memory & Registers:

```
# View registers
info registers
info registers all
print $r0
print $pc
print $sp

# Memory examination
x/10w 0x20000000    # 10 words at address
x/10b 0x08000000    # 10 bytes at address
```

```
x/10i $pc          # 10 instructions at PC
```

```
# Memory modification
set *(int*)0x20000000 = 0x12345678
set $r0 = 0x1000
```

Variables & Watchpoints:

```
# Print variables
print variable_name
print *pointer
print array[5]

# Set watchpoints
watch variable_name
watch *(int*)0x20000000

# Display (auto-print on stop)
display variable_name
undisplay 1
```

❏ Flash & Debug Using Monitor Commands

Complete Flash Sequence:

```
# 1. Connect to target
target remote localhost:3333

# 2. Halt and prepare
monitor reset halt

# 3. Unlock flash (if needed)
monitor stm32f4x unlock 0

# 4. Erase flash
monitor flash erase_sector 0 0 last
# or specific sectors
monitor flash erase_sector 0 0 7

# 5. Program flash
monitor flash write_image erase firmware.bin 0x08000000
# or
monitor program firmware.elf verify

# 6. Verify programming
monitor verify_image firmware.bin 0x08000000

# 7. Reset and run
monitor reset run
```

Step-by-Step Flash Commands:

```
# Method 1: Automatic (recommended)
monitor program firmware.elf verify reset

# Method 2: Manual control
monitor halt
monitor flash erase_address 0x08000000 0x10000 # Erase 64KB
```

```
monitor flash write_image firmware.bin 0x08000000
monitor verify_image firmware.bin 0x08000000
monitor reset run

# Method 3: Sector-specific
monitor halt
monitor flash info 0          # Show flash layout
monitor flash erase_sector 0 0 4  # Erase sectors 0-4
monitor flash write_image erase firmware.elf
monitor reset run
```

Debug Session Commands:

```
# Start debug session
arm-none-eabi-gdb firmware.elf
target remote :3333

# Load and prepare
load
monitor reset halt

# Set breakpoints for your blinky code
break main
break main.c:37          # LED toggle line
break gpio_pin_toggle_dt

# Start debugging
continue

# When halted, examine state
print led_state
print ret
print led
info registers

# Memory inspection
x/10w $sp          # Stack
x/10w 0x20000000    # RAM start
x/10i $pc          # Current instructions

# GPIO register inspection (STM32F4 example)
x/w 0x40020C00      # GPIOD_MODER
x/w 0x40020C14      # GPIOD_ODR (output data)

# Continue debugging
step
next
continue
```

❑ Power User Commands

Advanced Monitor Commands:

```
# Flash operations
monitor flash info 0
monitor flash list
monitor flash banks

# Memory operations
monitor mdw 0x08000000 10      # Read 10 words
```

```
monitor mdb 0x20000000 256      # Read 256 bytes
monitor mww 0x20000000 0x1234  # Write word

# Target control
monitor reg                      # Show all registers
monitor reg pc                   # Show PC register
monitor reg r0 0x1000           # Set R0 register

# Debug features
monitor arm semihosting enable
monitor tpiu config internal - uart off 2000000
```

Useful GDB Scripts:

```
# Auto-load and start debugging
define flash_and_debug
    monitor program firmware.elf verify reset
    load
    break main
    continue
end

# Reset and restart
define restart
    monitor reset halt
    load
    continue
end

# Memory dump
define memdump
    dump binary memory dump.bin 0x08000000 0x08010000
end
```

OpenOCD Telnet Interface:

```
# Connect to OpenOCD telnet (port 4444)
telnet localhost 4444

# Direct commands (no 'monitor' prefix needed)
> halt
> flash info 0
> reset run
> shutdown
```

□□ Target-Specific Examples

STM32F4 Discovery:

```
# OpenOCD startup
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg

# GDB connection
arm-none-eabi-gdb firmware.elf
(gdb) target remote :3333
(gdb) monitor program firmware.elf verify reset
(gdb) break main
(gdb) continue
```

Infineon CYT2CL7BAAQ (TRAVEO™ T2G Cluster):

```
# OpenOCD startup - Infineon TRAVEO T2G with dual Cortex-M7 cores
openocd -f interface/kitprog3.cfg -f target/psoc6.cfg -c "set CHIPNAME cyp" -c "set WORKAREAS

# Alternative with JLink (if using Segger probe)
openocd -f interface/jlink.cfg -f target/psoc6.cfg -c "adapter speed 2000"

# Specific target configuration for CYT2CL7BAAQ
# Create custom config file: cyp2cl7.cfg
echo '
source [find target/psoc6.cfg]
set CHIPNAME cyp2cl7baaq
set WORKAREASIZE 0x8000
adapter speed 2000

# Dual Cortex-M7 configuration
set _TARGETNAME_0 $_CHIPNAME.cm7_0
set _TARGETNAME_1 $_CHIPNAME.cm7_1

# CM7_0 (Primary core)
target create $_TARGETNAME_0 cortex_m -endian little -dap $_CHIPNAME.dap -ap-num 1 -coreid 0
$_TARGETNAME_0 configure -work-area-phys 0x08000000 -work-area-size $_WORKAREASIZE

# CM7_1 (Secondary core)
target create $_TARGETNAME_1 cortex_m -endian little -dap $_CHIPNAME.dap -ap-num 2 -coreid 1
$_TARGETNAME_1 configure -work-area-phys 0x08000000 -work-area-size $_WORKAREASIZE

# Flash configuration for CYT2CL7BAAQ (8MB Flash)
flash bank $_CHIPNAME.flash_cm7_0 psoc6 0x10000000 0x800000 0 0 $_TARGETNAME_0
flash bank $_CHIPNAME.flash_cm7_1 psoc6 0x10400000 0x400000 0 0 $_TARGETNAME_1

targets $_TARGETNAME_0
' > cyp2cl7.cfg

# Launch with custom config
openocd -f interface/kitprog3.cfg -f cyp2cl7.cfg

# GDB connection (Primary CM7 core)
arm-none-eabi-gdb firmware.elf
(gdb) target remote :3333
(gdb) monitor psoc6 reset_halt
(gdb) load
(gdb) break main
(gdb) continue

# For dual-core debugging (connect to secondary core on port 3334)
arm-none-eabi-gdb firmware_cm7_1.elf
(gdb) target remote :3334
```

CYT2CL7BAAQ Flash Programming:

```
# Flash CM7_0 (Primary core)
(gdb) monitor halt
(gdb) monitor flash erase_address 0x10000000 0x100000 # Erase 1MB
(gdb) monitor flash write_image erase firmware_cm7_0.hex
(gdb) monitor verify_image firmware_cm7_0.hex 0x10000000

# Flash CM7_1 (Secondary core)
(gdb) monitor halt
(gdb) monitor flash erase_address 0x10400000 0x80000 # Erase 512KB
(gdb) monitor flash write_image erase firmware_cm7_1.hex 0x10400000
```

```
(gdb) monitor verify_image firmware_cm7_1.hex 0x10400000
```

```
# Reset both cores
```

```
(gdb) monitor psoc6 reset_halt
```

```
(gdb) monitor psoc6 reset_run
```

CYT2CL7BAAQ Memory Map:

Flash Memory:

- CM7_0 Code Flash: 0x10000000 - 0x107FFFFFFF (8MB)
- CM7_1 Code Flash: 0x10400000 - 0x107FFFFFFF (4MB shared)
- Work Flash: 0x14000000 - 0x14007FFF (32KB)

SRAM Memory:

- CM7_0 SRAM: 0x28000000 - 0x2807FFFF (512KB)
- CM7_1 SRAM: 0x28080000 - 0x280BFFFF (256KB)
- Shared SRAM: 0x280C0000 - 0x280FFFFFFF (256KB)

Peripherals:

- CM7_0 Peripherals: 0x40000000 - 0x4FFFFFFF
- CM7_1 Peripherals: 0x42000000 - 0x4FFFFFFF

Generic Cortex-M Setup:

```
# For any Cortex-M with SWD
```

```
openocd -c "interface stlink; transport select hla_swd; set CHIPNAME stm32f4x; source [find t.
```

❏ Zephyr RTOS Support for CYT2CL7BAAQ

Board Configuration:

```
# In your Zephyr project: boards/arm/cyp2cl7baaq/cyp2cl7baaq.dts
/dts-v1/;
```

```
/ {
    model = "Infineon CYT2CL7BAAQ TRAVEO T2G Cluster";
    compatible = "infineon,cyp2cl7baaq";

    chosen {
        zephyr,console = &uart0;
        zephyr,shell-uart = &uart0;
        zephyr,flash = &flash0;
        zephyr,sram = &sram0;
    };

    cpus {
        cpu@0 {
            device_type = "cpu";
            compatible = "arm,cortex-m7";
        };
        cpu@1 {
            device_type = "cpu";
            compatible = "arm,cortex-m7";
        };
    };

    flash0: flash@10000000 {
        compatible = "soc-nv-flash";
    };
};
```

```

        reg = <0x10000000 DT_SIZE_M(8)>;
    };

    sram0: memory@28000000 {
        compatible = "mmio-sram";
        reg = <0x28000000 DT_SIZE_K(512)>;
    };
};

```

West Commands for CYT2CL7BAAQ:

```

# Build for CYT2CL7BAAQ
west build -p always -b cyp2cl7baaq samples/basic/blink

# Flash using OpenOCD
west flash --runner openocd

# Debug with west
west debug --runner openocd

# Debug server (background)
west debugserver --runner openocd

```

Custom OpenOCD Runner Configuration:

```

# In your project: boards/arm/cyp2cl7baaq/board.cmake
board_runner_args(openocd --config=${BOARD_DIR}/support/openocd.cfg)

include(${ZEPHYR_BASE}/boards/common/openocd.board.cmake)

```

OpenOCD Configuration for Zephyr:

```

# boards/arm/cyp2cl7baaq/support/openocd.cfg
source [find interface/kitprog3.cfg]

transport select swd
adapter speed 2000

source [find target/psoc6.cfg]

# Override for CYT2CL7BAAQ specifics
set CHIPNAME cyp2cl7baaq
set WORKAREASIZE 0x8000

# Zephyr-specific flash configuration
$_TARGETNAME configure -event reset-init {
    # Configure clocks for flash programming
    mww 0x40260200 0x01 # Enable HFCLK
    mww 0x40200008 0x00 # Configure flash wait states
}

# Flash bank for Zephyr
flash bank zephyr.flash psoc6 0x10000000 0x800000 0 0 $_TARGETNAME

```

📁 Essential File Structure

Bare Metal Project:

```

project/
├── src/
│   ├── main.c           # Your blinky code
│   └── startup.s        # Startup assembly
├── include/
├── linker_script.ld     # Memory layout
├── Makefile             # Build rules
└── openocd.cfg          # OpenOCD config

```

Zephyr RTOS Project:

```

zephyr_project/
├── boards/
│   └── arm/
│       └── cyp2cl7baaq/  # Board definition
├── src/
│   └── main.c            # Application code
├── prj.conf              # Kernel config
├── CMakeLists.txt        # Build system
└── .vscode/
    ├── launch.json       # Debug configs
    └── tasks.json        # Build tasks

```

Sample openocd.cfg:

```

source [find interface/stlink.cfg]
source [find target/stm32f4x.cfg]
adapter speed 2000

```

Common Issues & Solutions

Connection Issues:

```

# Check if OpenOCD can detect your probe
openocd -f interface/stlink.cfg -c "init; exit"

# Reset USB connection
# Unplug and replug ST-Link USB

# Check permissions (Linux/macOS)
sudo openocd -f interface/stlink.cfg -f target/stm32f4x.cfg

```

Flash Programming Issues:

```

# If flash is protected
monitor stm32f4x unlock 0
monitor stm32f4x mass_erase 0

# If programming fails
monitor reset halt
monitor flash erase_sector 0 0 last
monitor program firmware.elf verify reset

```

Debugging Issues:

```
# If breakpoints don't work
monitor reset halt
load
break main
continue

# If symbols are missing
file firmware.elf
symbol-file firmware.elf

# If execution seems stuck
monitor halt
info registers
x/10i $pc
```

☐☐ Quick Command Reference Table

Operation	GDB Command	Monitor Command	Description
Connect	target remote :3333	-	Connect to OpenOCD
Load	load	-	Load ELF to target
Flash	-	program firmware.elf verify reset	Flash ELF file
Halt	Ctrl+C	halt	Stop execution
Reset	-	reset halt	Reset and halt
Resume	continue	resume	Continue execution
Breakpoint	break main	-	Set breakpoint
Step	step / next	-	Single step
Registers	info registers	reg	Show registers
Memory	x/10w 0x20000000	mdw 0x20000000 10	Examine memory
Quit	quit	shutdown	Exit debugger

☐☐ Pro Tips

1. **Always halt before flashing:** `monitor reset halt`
2. **Use `load` after `monitor program`:** Ensures symbols are loaded
3. **Set breakpoints before `continue`:** More reliable than setting during execution
4. **Use `info registers` frequently:** Monitor CPU state
5. **Check memory alignment:** ARM requires aligned access for words
6. **Use `monitor` prefix in GDB:** For OpenOCD commands
7. **Start OpenOCD first:** Then connect GDB
8. **Use `.gdbinit`:** Automate common setup commands

This cheatsheet covers everything you need for ARM embedded development with OpenOCD and GDB! Save it for quick reference during your debugging sessions. ☐☐